Finshot — Backend Developer Take-Home Assignment

Version 3.11 · Remittance Transfer API

Read this first

Part 1	Required. About **3 hours**.
Part 2	Optional. About **3 hours**. Skipping it does not disqualify you.
Deadline	4 calendar days
Scope	Backend only. **Do not build a UI.**
Language	Java 17+ with Spring Boot preferred. Kotlin, Go, Node.js, PHP also accepted.
Database	H2 in-memory or SQLite. We must not install anything.

You can pass with Part 1 alone. Part 1 done well and explained clearly scores higher than Part 2 done badly. Do not rush Part 1 to reach Part 2.

About AI tools

AI tools are **allowed**. We use them at work every day.

One rule: in `README.md`, write 3 lines about which tools you used and where. This is not scored. Not disclosing it, if we find out later, is.

In the interview we will share your screen and ask you to explain, change, and extend your own code live. Submit only code you can defend.

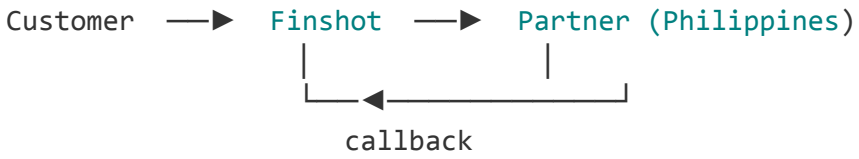
About English

This brief is in English because you will read English specifications every day in this role. Your team's specifications, tickets, and code comments are all in English.

- **We do not score your grammar or writing style.** We score what you say, not how you say it.
- Short and simple English is fine. Preferred, actually.
- **If anything is unclear, email us in Indonesian.** Asking is not a penalty.

The domain

A customer in Korea sends money to the Philippines.



We record the request, call an external partner, and later receive a callback with the result.

PART 1 — Required (about 3 hours)

How to spend the 3 hours

Code	about 2 hours
`RESULT.md`, `DESIGN.md`, `README.md`	about 1 hour

`RESULT.md` and `DESIGN.md` carry **35 of the 70 Part 1 points**. Do not leave them for the last ten minutes.

You do not need to build any of the following. We give no points for them:

- authentication, authorization, or user accounts
- list endpoints, pagination, or filtering
- Docker, CI, OpenAPI/Swagger, or logging framework setup
- an admin API, metrics, or health checks
- your own error framework — use the shape we give you in 1.3
- anything from Part 2, until Part 1 works

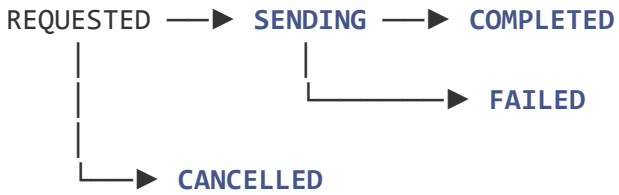
1.1 Calculation rules

Rule	**Value**
Fee	1% of `sendAmount`, minimum 3,000 KRW
Fee rounding	Whole KRW, HALF_UP
FX rate	1 KRW = 0.0412 PHP (hard-code this)
`receiveAmount`	`sendAmount × 0.0412`, 2 decimals, HALF_UP
`totalDebit`	`sendAmount + fee`

Example: `sendAmount` = 500000 → fee 5000, totalDebit 505000, receiveAmount 20600.00

Every amount must be exact. `receiveAmount` must always be returned with exactly 2 decimal places, including trailing zeros. How you represent money in your code, your API, and your database is your decision — explain it in `DESIGN.md` Q3.

1.2 States



Any other transition → **409**.

In Part 1, a transfer stays in `REQUESTED` after creation. You do not need to call a partner yet.

1.3 API

Create

These are the fields we will always send. **This is a minimum, not a closed list** — you may require additional headers or additional body fields, as long as you document them.

```
POST /api/transfers
{
  "customerId": "C001",
  "sendCurrency": "KRW",
  "sendAmount": "500000",
  "receiveCurrency": "PHP",
  "recipientName": "Juan Dela Cruz"
}
```

201 Created

```
{
  "transferId": "...",
  "status": "REQUESTED",
  "sendAmount": "500000",
  "fee": "5000",
  "totalDebit": "505000",
  "receiveAmount": "20600.00",
  "createdAt": "..."
}
```

The response must contain at least these fields, with these names and these values. You may add more.

This endpoint must be idempotent

A client will retry a create request — after a timeout, a dropped connection, or a user tapping the button twice. **A retry must never create a second transfer.**

We do not tell you how to do this. Design it yourself:

- Decide how a client tells you that a request is a retry of an earlier one. You may add a request header, add a field to the request body, or derive it from the request — your choice.
- Decide what happens when that same retry signal arrives with **different** request data.

- Decide what happens if your mechanism needs something from the client and the client does not send it.

Required behaviour:

Situation	Required behaviour
The same create request is retried	Return the original transfer. **Do not create a second one.**
A retry signal reused with different request data	**409**

Document your mechanism in `README.md`: what you added to the request, what a client must send, and how a client should generate it. One short paragraph is enough — but it must be enough for us to reproduce your calls. Explain the design in `DESIGN.md` Q2.

Read

```
GET /api/transfers/{transferId}
```

→ **200**, or **404** if not found.

Cancel

```
POST /api/transfers/{transferId}/cancel
```

Allowed only from `REQUESTED`. Otherwise **409**.

Error responses

Every non-2xx response uses this shape. **Do not design your own error model** — we are not scoring it, and building one costs you time you need elsewhere.

```
{
  "error": "VALIDATION_ERROR",
  "message": "sendAmount must be at least 10000"
}
```

`error` values: `VALIDATION_ERROR` (400), `NOT_FOUND` (404), `CONFLICT` (409). Part 2 adds `LIMIT_EXCEEDED` (422). The `message` is free text for humans — we do not parse it.

Validation → 400

- `sendAmount` ≤ 0, not a number, or has decimals
- `sendAmount` < 10,000
- `recipientName` empty or over 100 characters
- `sendCurrency` ≠ "KRW", or `receiveCurrency` ≠ "PHP"
- unknown `customerId`

1.4 schema.sql

You design the schema. We score it (15 points) and ask you about it in `DESIGN.md` Q1.

One table is given, so that every submission starts from the same customers:

```
CREATE TABLE customers (
  customer_id VARCHAR(10) PRIMARY KEY,
  name        VARCHAR(100) NOT NULL
);
INSERT INTO customers (customer_id, name) VALUES
  ('C001', 'Kim'), ('C002', 'Lee'), ('C003', 'Park');
```

Everything else is yours to design.

1.5 Required scenarios — S1 to S6

Run these in order on a freshly started app. Paste request, status code, and response body into `RESULT.md`.

Show the **complete** request — headers and body. We must be able to reproduce your calls exactly.

To save time, you may paste a `curl` script and its captured output instead of writing each block by hand. Either way is fine — we only need the request, the status code, and the response body.

#	Scenario
S1	Create a transfer for `C001`, amount 500000

#	Scenario
S2	Retry S1 — assume the response never arrived, and send the exact same request again, **including whatever signals a retry**
S3	Send again with the **same retry signal** as S1, but amount 600000
S4	Create another transfer, then cancel it. Show both responses.
S5	Cancel the same transfer again
S6	Create with `sendAmount` = 5000

Format for each:

S1

Request:

```
POST /api/transfers
<headers>
{ "customerId": "C001", ... }
```

Status: 201

Response:

```
{ ... }
```

1.6 Tests — optional

Tests are not required. Not writing them is not a deduction, and we do not measure coverage.

If you do write them, save the console output of your test run as `test-output.txt`. Two meaningful tests tell us more than a large suite.

PART 2 — Optional (about 3 hours)

Attempt this only after Part 1 is complete and working.

2.1 Partner client

Implement a `PartnerClient` yourself. Do not call any real service.

It must be deterministic:

fails when `abs(partnerRef.hashCode()) % 10 < 3`

- When a transfer enters SENDING, you generate a `partnerRef` (a string you create and store on the transfer). The same `partnerRef` is used for all retry attempts of that transfer. The PartnerClient must be a pure function of `partnerRef` so runs are reproducible: **it fails when the last digit of `partnerRef` is 0, 1, or 2, and succeeds otherwise.** For S9, choose a `partnerRef` that ends in 0–2.

succeeds otherwise

Do not use a random number generator. Each attempt must take at least 200 ms.

What "success" means

A successful partner call means the partner has **accepted the request**. It does not mean the money reached the recipient.

A transfer must never become `COMPLETED` because the partner call succeeded. It stays in `SENDING` until the partner callback (2.3) tells us the final result.

Flow:

1. Transfer moves to `SENDING`
2. Call the partner — retry up to **3 attempts total** with backoff
3. **Partner accepts** → transfer **stays in `SENDING`**, waiting for the callback
4. **All 3 attempts fail** → transfer becomes `FAILED` with a reason

Log every attempt: attempt number, `partnerRef`, outcome.

2.2 Daily limit

- Maximum **3,000,000 KRW** of `sendAmount` per customer per day

- Day boundary: **Asia/Seoul (UTC+9)**
- `CANCELLED` and `FAILED` transfers do not count
- Exceeding the limit → **422**

Two requests arriving at the same instant must not both pass when together they would exceed the limit.

2.3 Partner callback

```
POST /api/callbacks/partner
{
  "partnerRef": "...",
  "eventId": "evt_0001",
  "status": "COMPLETED"
}
```

A callback is the **only** way a transfer becomes `COMPLETED`.

- **`status` is always `COMPLETED`.** To keep this exercise small, the partner never sends `FAILED` or `CANCELLED` in a callback. You do not need to handle those values.
- A transfer becomes `FAILED` only when all 3 partner attempts fail (see 2.1).
- Valid only from `SENDING`. Otherwise **409**.
- **Idempotent by `eventId`.** The same `eventId` twice must not change state twice, and must not error on the second delivery.
- Unknown `partnerRef` → **404**

2.4 Optional scenarios — S7 to S9

#	Scenario
S7	With `C002`, submit transfers until the daily limit is exceeded. Show the last success and the rejection.
S8	Send the same callback (same `eventId`) twice. Show both responses and the final state.
S9	Create a transfer whose `partnerRef` makes the partner fail. Show the retry log and the final state.

What to submit

Use **exactly** these file names at the repository root.

```
/README.md          -- how to run (ONE command), stack used, how a client
retries
                       a create request safely, AI tools used,
                       what you did not implement
/DESIGN.md          -- answers to the questions below
/schema.sql         -- CREATE TABLE, indexes, constraints, seed data (see
1.4)
/RESULT.md          -- S1-S6 (and S7-S9 if you did Part 2)
/src/...            -- source code
/test-output.txt    -- OPTIONAL. Only if you wrote tests (see 1.6).
```

Delete `target/`, `build/`, `node_modules/`, `vendor/`, `.git/`. Submit a `.zip`.

DESIGN.md

Answer in **150 words or fewer** each. Short and specific is better than long and general.

Grammar is not scored. Write simple English.

Part 1 questions — required

> **Q1.** Describe your tables. Why these indexes and constraints? Which constraint stops which problem?
>
> **Q2.** How does your idempotency work? What identifies a retry, and where is it enforced — application code, database, or somewhere else? What happens if two identical create requests arrive at the exact same moment?
>
> **Q3.** How do you represent, store, and round money — in your code, your API, and your database? At which exact step does rounding happen, and why there?

Part 2 questions — only if you did Part 2

> **Q4.** How did you make the daily limit safe when requests arrive at the same time? Name the mechanism you used (row lock, unique constraint, isolation level, something else) and what you gave up by choosing it.
>
> **Q5.** The partner callback arrives **before** your code finished saving the result of the partner call. What happens in your code? **If it breaks, say so** and explain how you would fix it.

A note on Q5: answering "my implementation breaks here, and here is how I would fix it" earns full marks. Noticing the problem is the point.

Honesty

If a scenario does not work, **write what actually happened** in `RESULT.md`.

We run the code for shortlisted candidates. An honest failure report scores better than a fake success — and we would rather hire someone who reports problems clearly.

If a requirement is unclear, make a reasonable assumption, implement it, and note the assumption in `README.md`. Spotting an ambiguity and closing it is a large part of this job.

Good luck.